Reproducible Documentation Pipelines

A whitepaper on deterministic, secure document generation

md2pdf project — 2026

Contents

Abstract

Documentation is increasingly generated in CI from plain-text sources, yet the tools that turn that text into distributable PDFs are rarely reproducible or sandboxed. We describe the properties a documentation pipeline needs to be trustworthy — determinism, hermeticity and bounded resource use — and show how a single-binary, network-free renderer achieves them.[1]

1. The problem

A "Markdown to PDF" step looks trivial until it runs a thousand times a day in CI. In practice it commonly pulls in:

- a headless browser (hundreds of MB, frequent CVEs), or
- a full TeX distribution (gigabytes, environment-sensitive output), or
- a Node/Python runtime with a deep, fast-moving dependency tree.

Each adds attack surface and non-determinism: the same Markdown can yield different bytes on different machines, defeating caching and supply-chain verification.

2. Properties of a trustworthy pipeline

Property	Definition
Deterministic	Same input → same output bytes, everywhere
Hermetic	No network access during rendering

[1]This document was itself produced by the pipeline it describes.

Property	Definition
Bounded	Inputs and assets are size-limited
Least-privilege	No path traversal; untrusted input cannot read the host

3. Design

A renderer-agnostic intermediate representation separates parsing from output. The default renderer compiles in-process, embeds its fonts, and is built without any network capability. Assets are resolved under a fixed root and denied otherwise.

```
Markdown → IR → Renderer → bytes      (fonts embedded, network off)
```

4. Results

Embedding fonts and compiling in-process yields byte-stable PDFs and millisecond cold starts, with a single dependency-free binary suitable for air-gapped CI.

5. Conclusion

Treating document generation as a build step — deterministic, hermetic, bounded — turns documentation from a fragile afterthought into a verifiable artifact.